

Introduction Big Data Processing

www.3aayaam.in

1

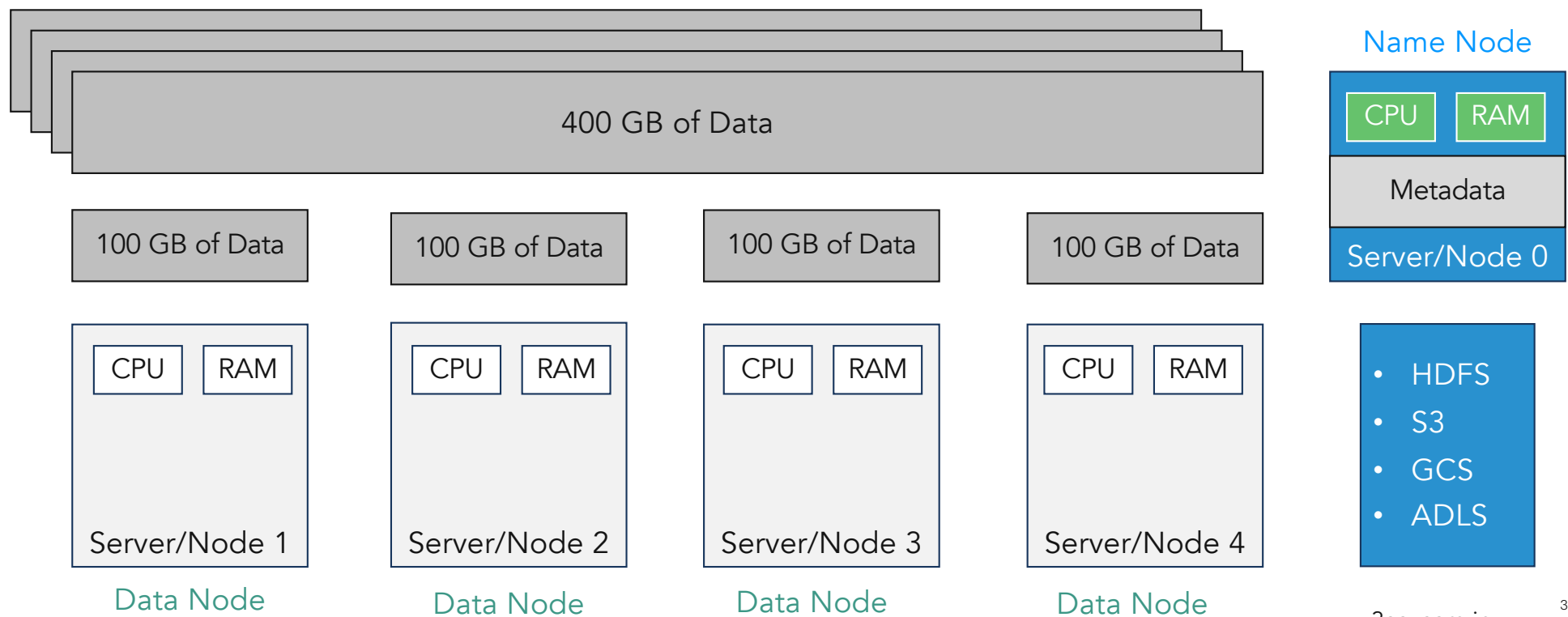


Agenda

1. Distributed Storage & Compute
2. Big Data
3. Hadoop – HDFS & MapReduce
4. File Formats – Parquet & Avro
5. YARN
6. ZooKeeper

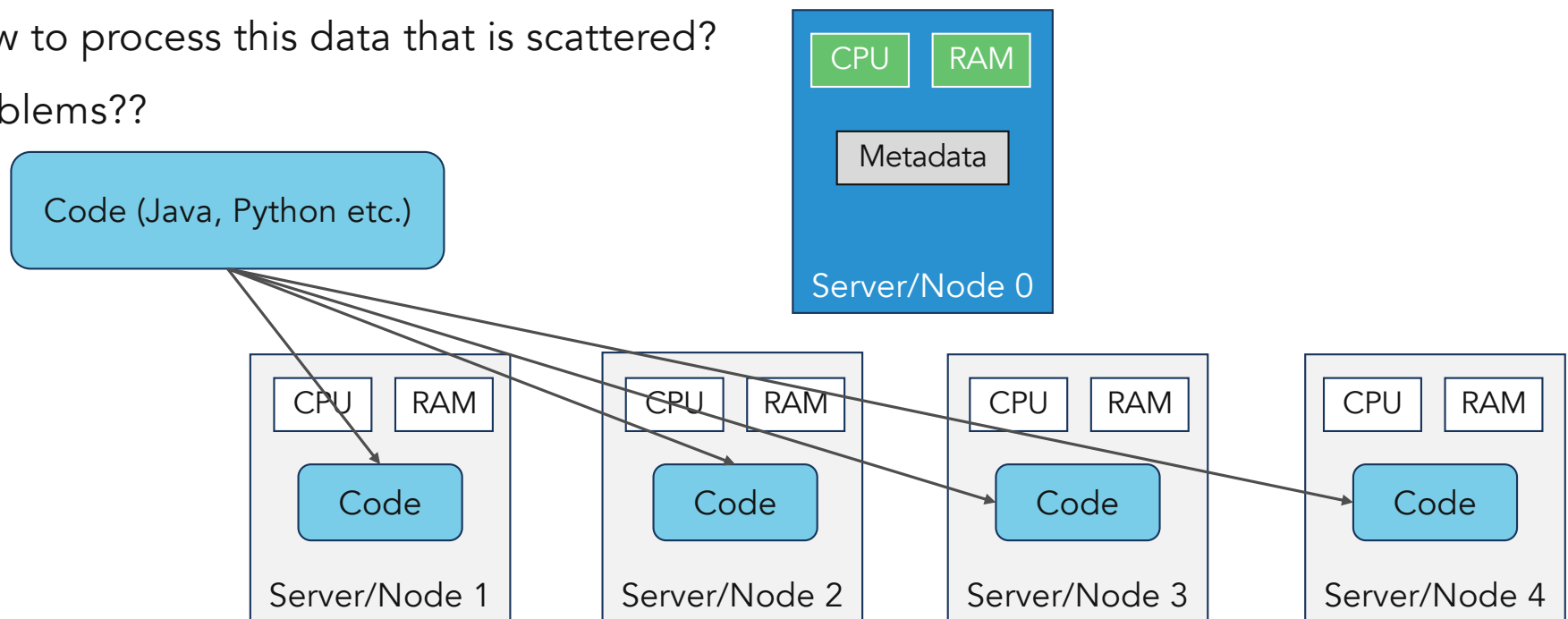
Distributed Storage

- Store data across multiple servers or nodes.



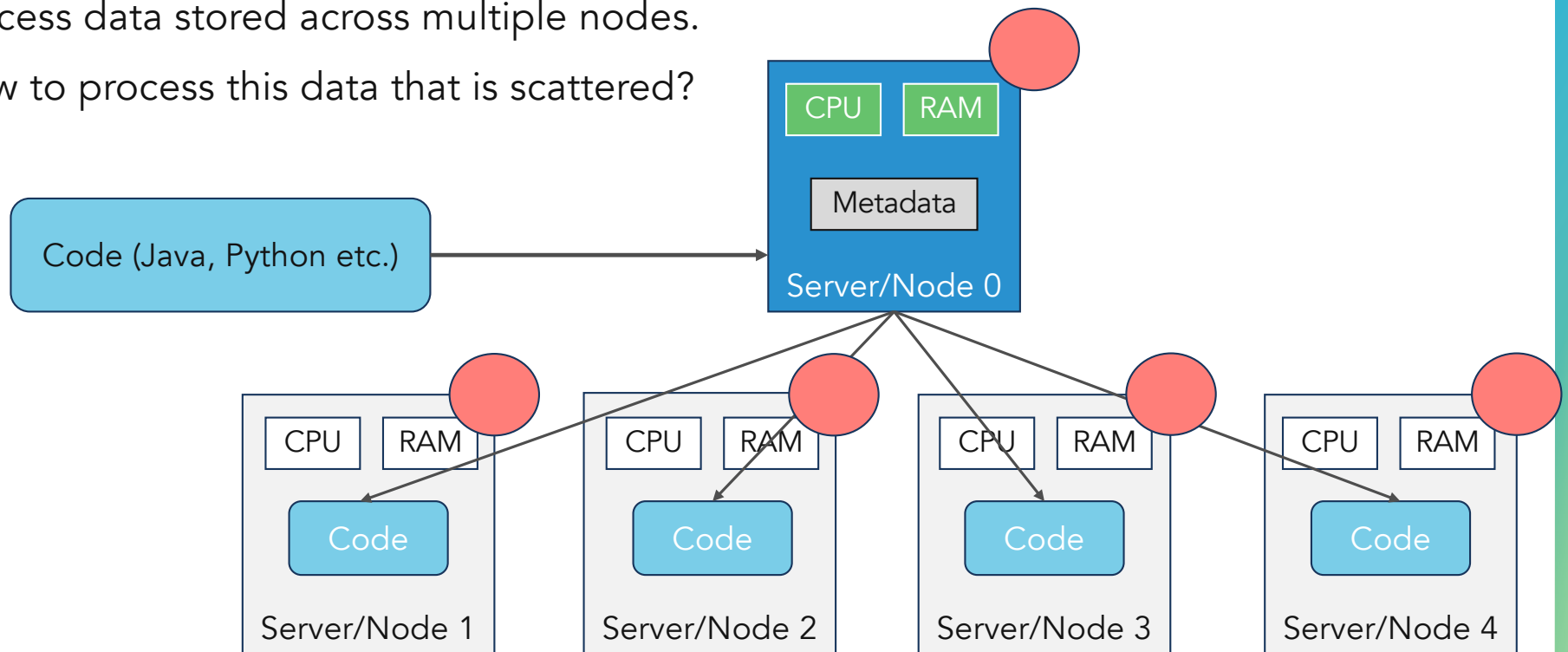
Distributed Compute

- Process data stored across multiple nodes.
- How to process this data that is scattered?
- Problems??



Distributed Compute

- Process data stored across multiple nodes.
- How to process this data that is scattered?



Big Data

- Data that needs distributed storage for persistence and distributed compute for processing.
- 5 Vs of Big Data
 - (Volume) Size of data. Hundreds of GBs to TBs, PBs and more. [Distributed Storage](#).
 - (Velocity) Data generation rate and speed. [Stream processing](#).
 - (Variety) Types of data. [Structured](#) (RDBMS, CSV, Parquet etc.), [Semi-structured](#) (NoSQL, JSON). [Unstructured](#) (videos, images).
 - Veracity & Value.

Big Data Technologies

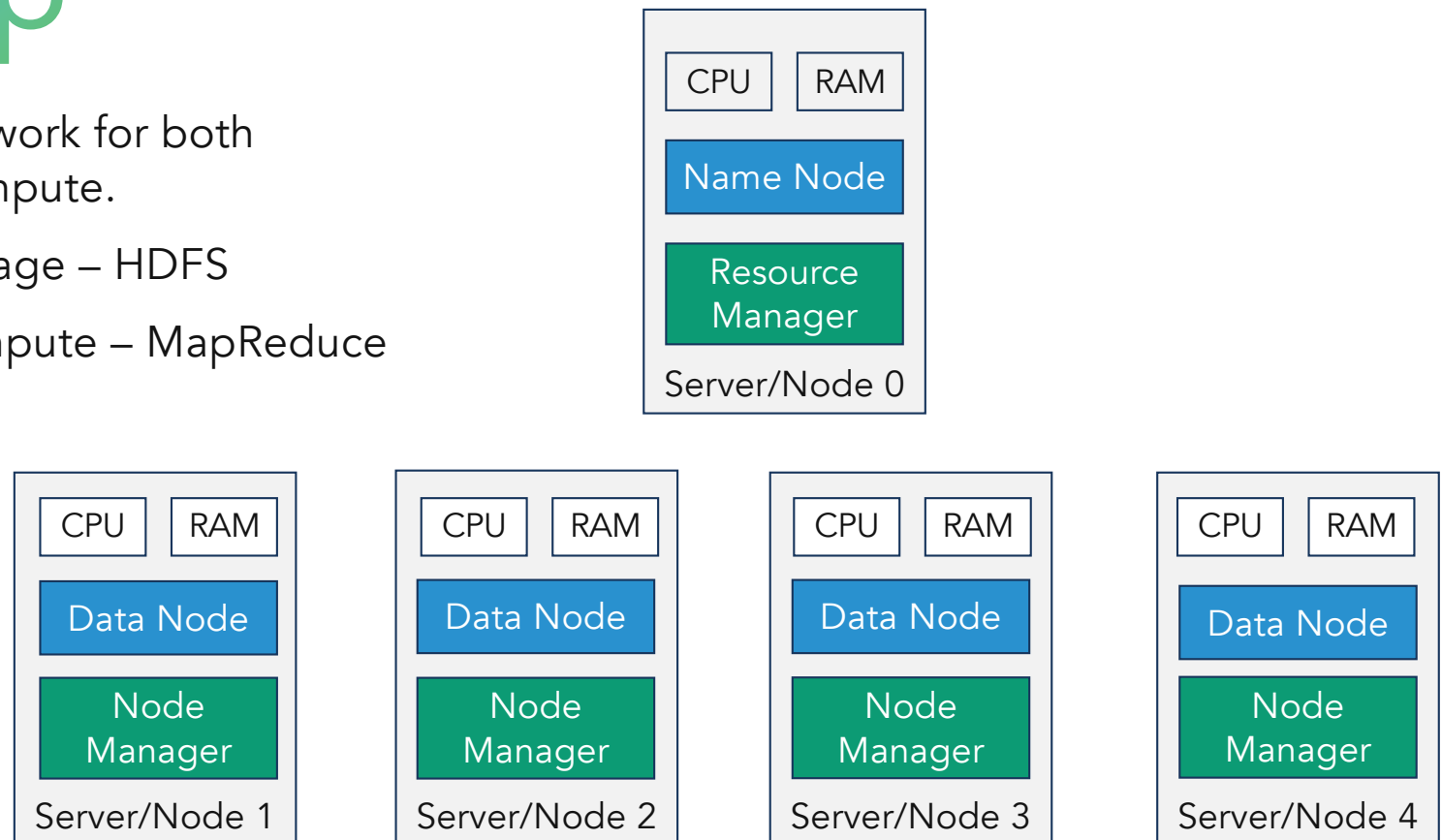


- Apache Spark – Batch & Stream Processing
- Apache Kafka – Streaming Ingestion
- Apache Flink – Stream Processing
- Kafka Streams – Stream Processing
- Apache Hadoop



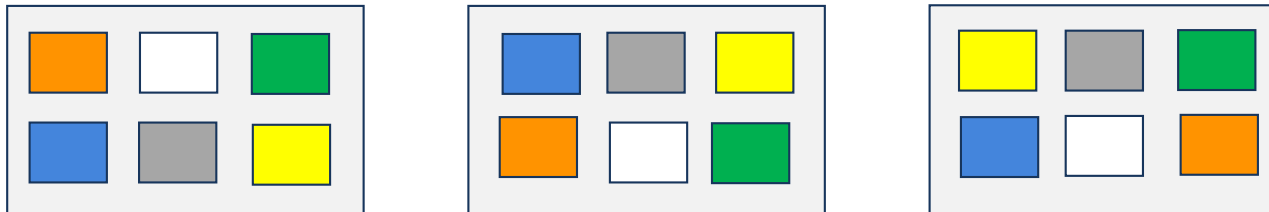
Hadoop

- Big Data Framework for both storage and compute.
- Distributed Storage – HDFS
- Distributed Compute – MapReduce



HDFS Storage

- IO – Input/Output operation. Read and Write operations.
- IO Blocks
- HDFS Block Size – 128 MB.
 - Record size – 500 Bytes. 1 HDFS block would contain 260,000+ records.
 - Record size – 2 KB. 1 HDFS block would contain 65000+ records.
- AWS S3, Google Cloud Storage, Azure Data Lake Storage

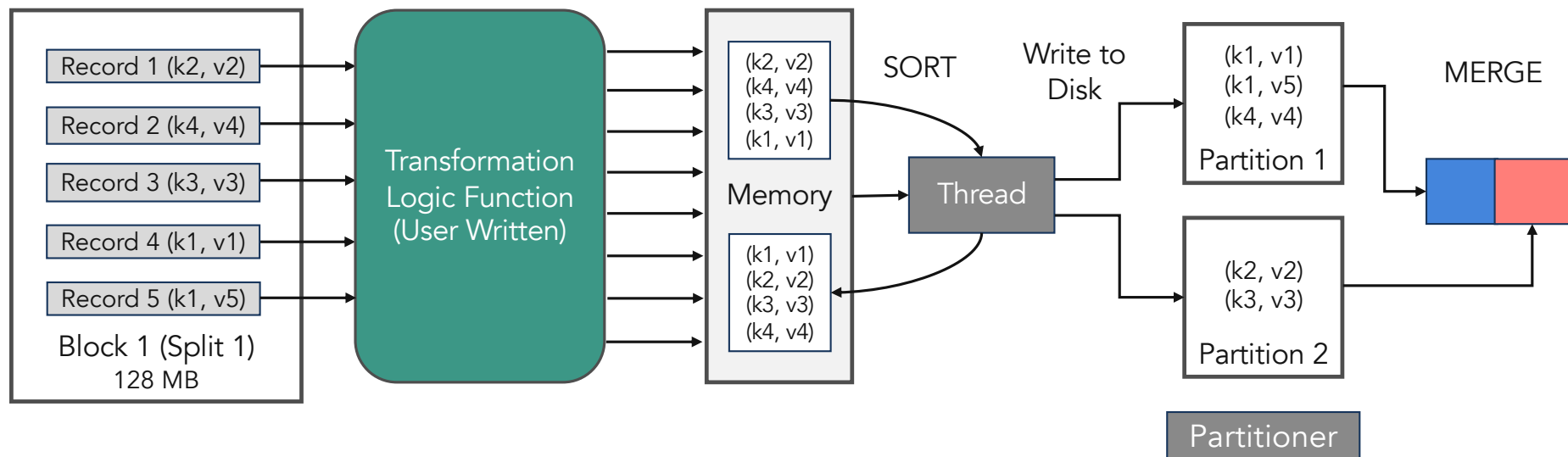


MapReduce

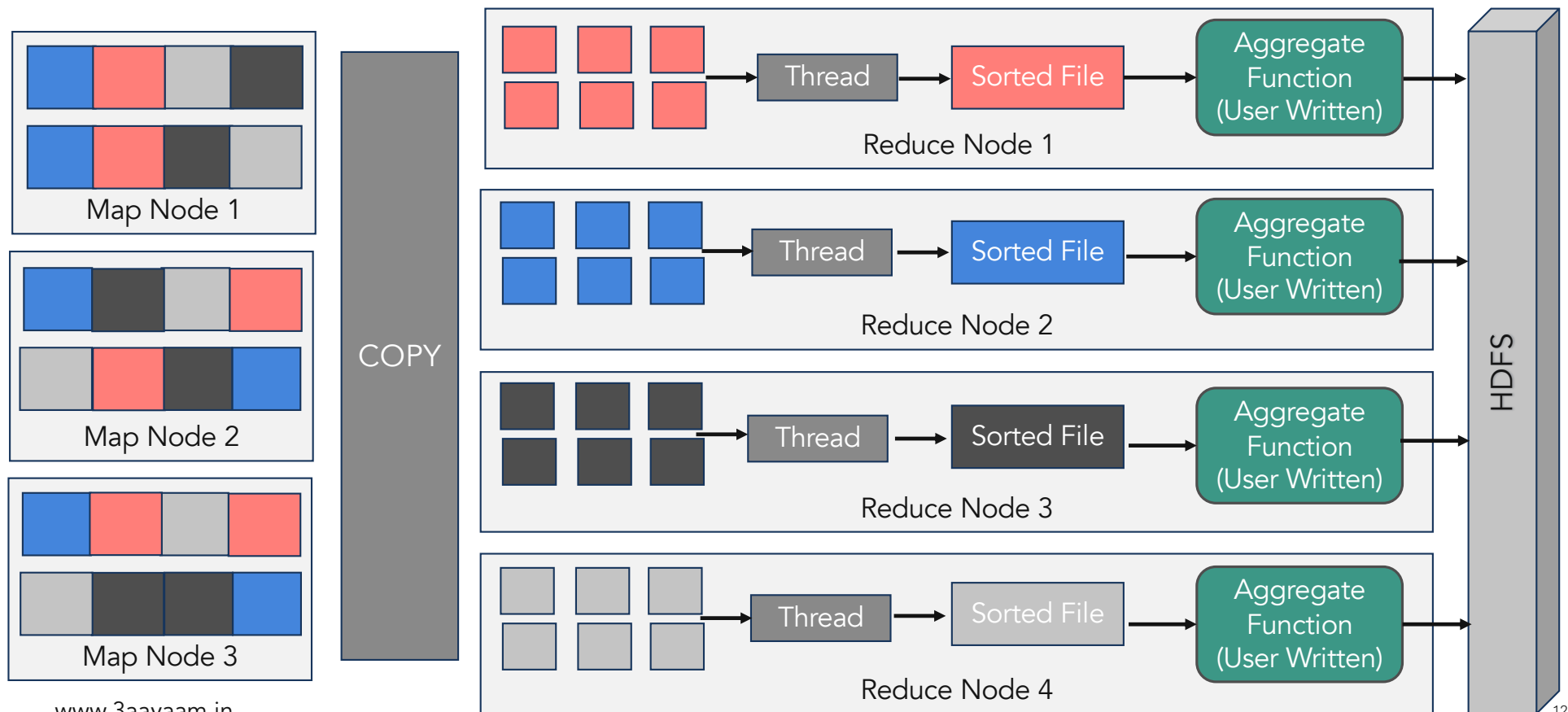
- Framework for Distributed Computing on Big Data that is stored on HDFS or other distributed storages.
- Map Task
- Reduce Task

Map "Phase"

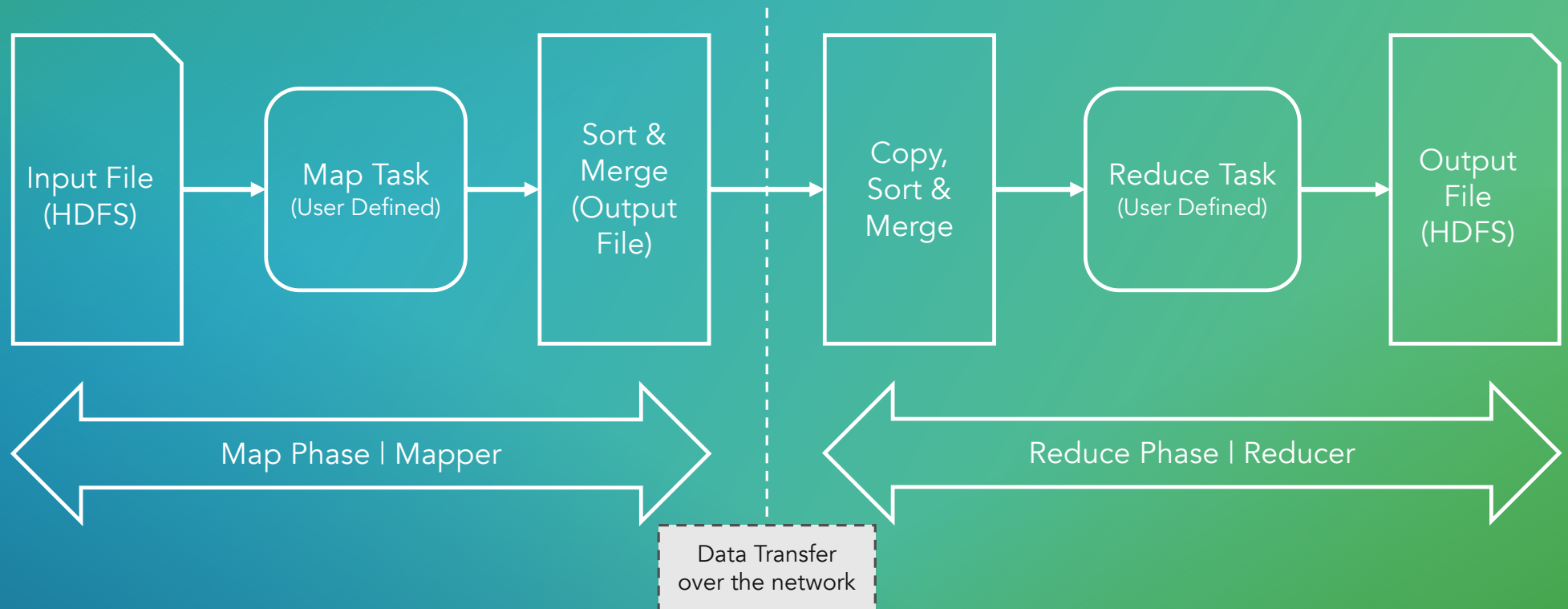
Mapper, Map Task, Map Partitions
SELECT, WHERE (Filter), Functions (CAST, DATE) on Rows



Reduce "Phase"



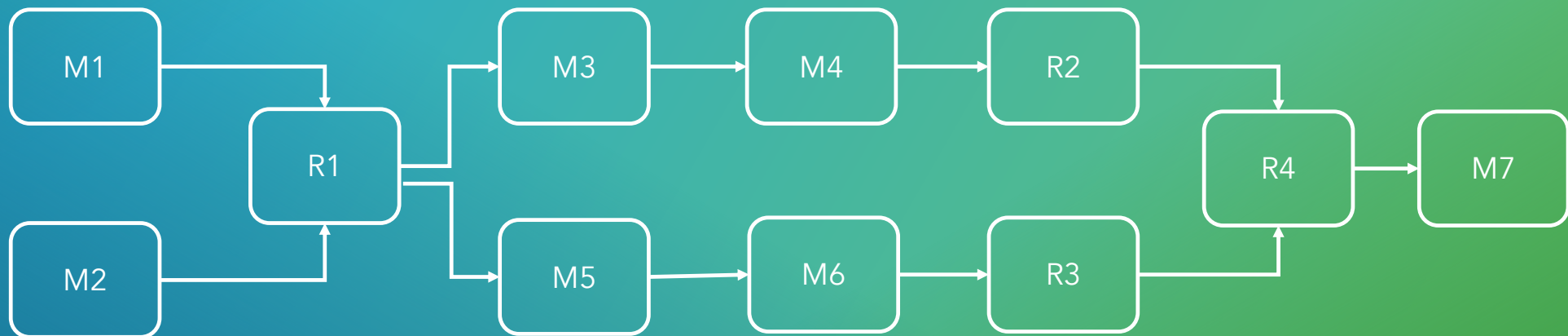
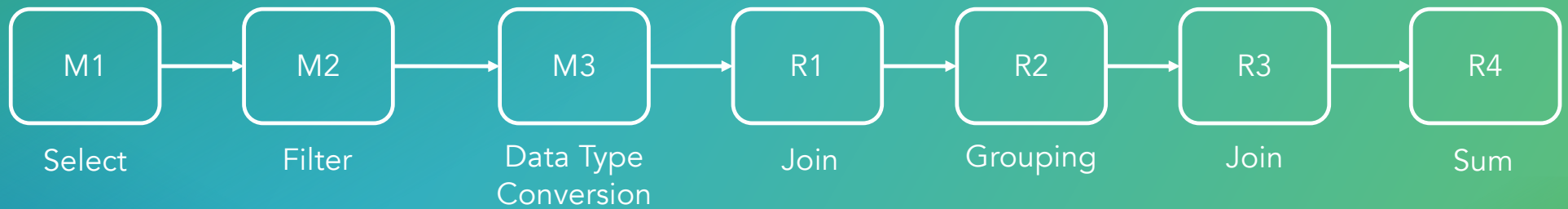
MapReduce Complete Flow



MapReduce Parallelism

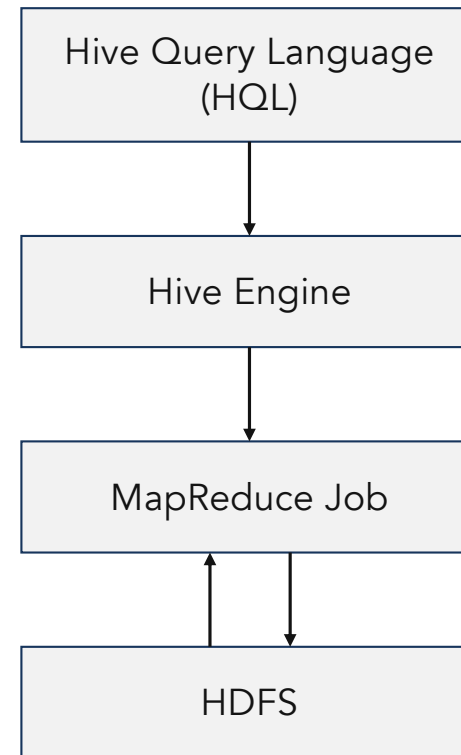


MapReduce Combination



Tools on Top of MapReduce

- Problem of MapReduce – Only Java Code.
- Hive – SQL like query on MapReduce.
- Pig – Scripting for ETL on MapReduce.
- Mahout – Machine Learning Library using MR.



Big Data (Hadoop) Ecosystem

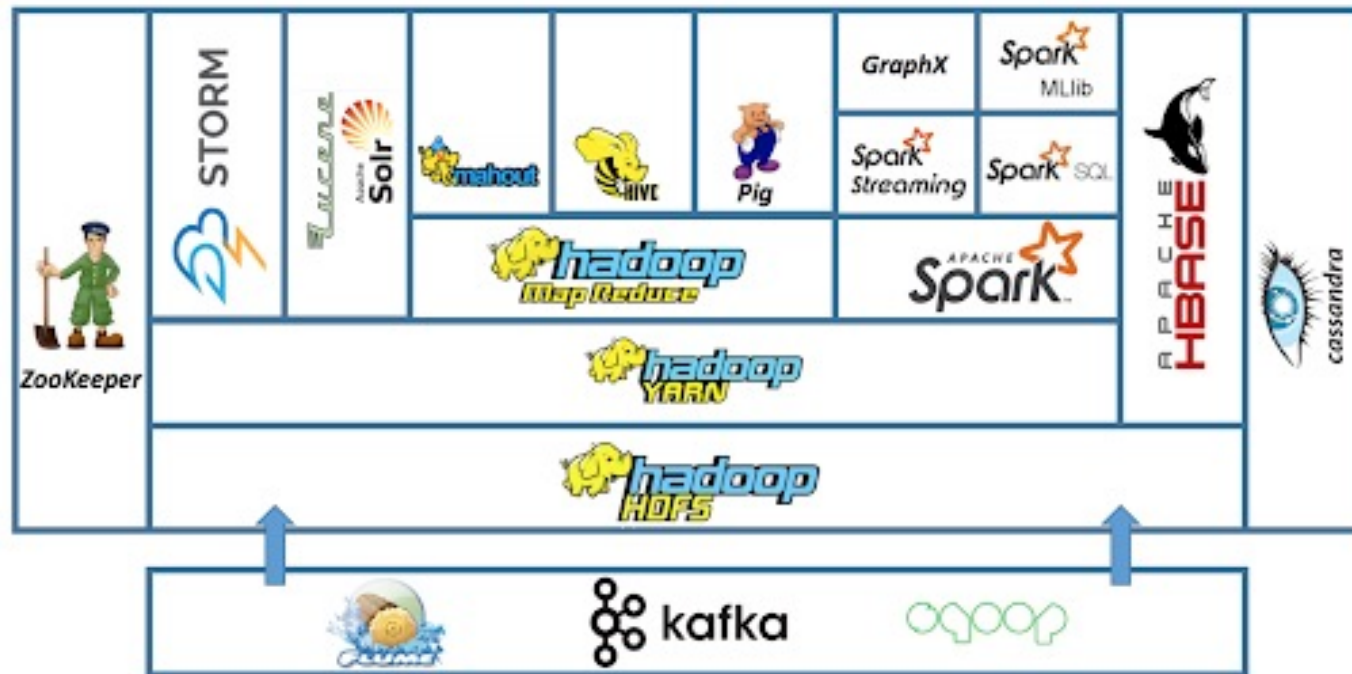
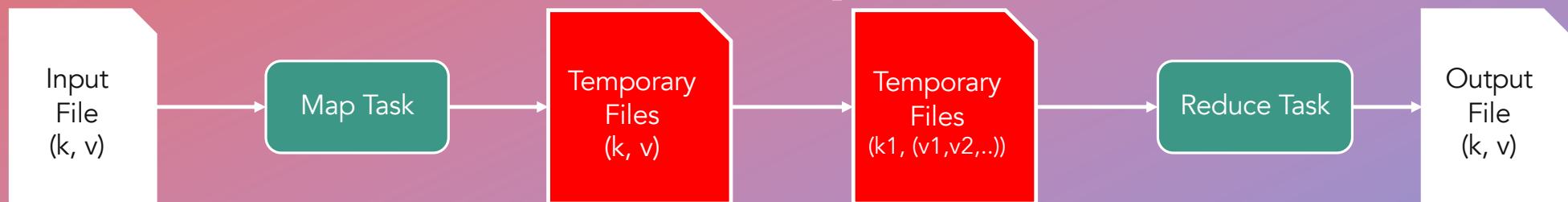


Image Courtesy: Google Search

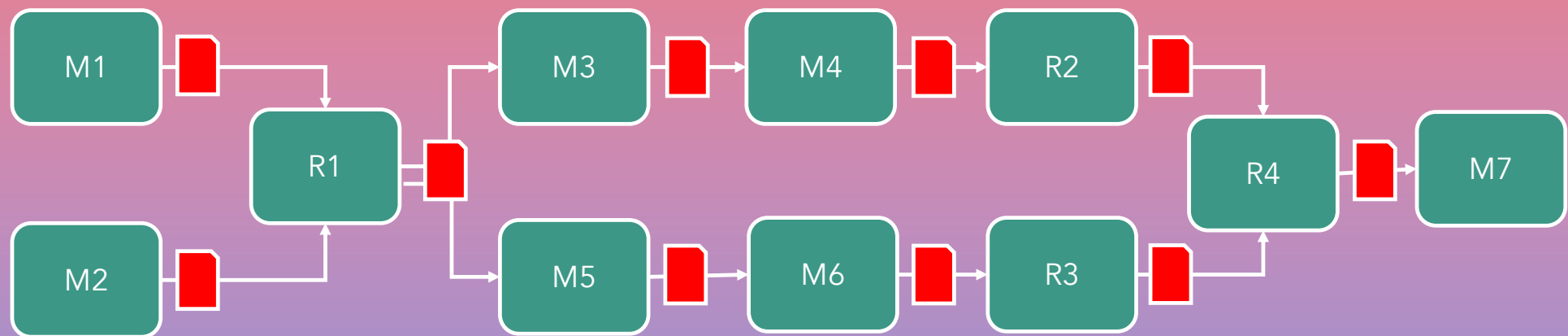
Hands On – MapReduce



- pip3 install mrjob. pip3 install setuptools.
- 1st Logic
 - Mapper – Define Key and Value.
 - Reducer – Count no. of passengers by country.
- 2nd Logic
 - Mapper – Define Key and Value
 - Reducer – Total spend by country.
- 3rd Logic
 - Mapper – Define Key and Value.
 - Mapper – Select records only from a specific country.

YOU DON'T HAVE TO DO THIS !!!!

MapReduce IO



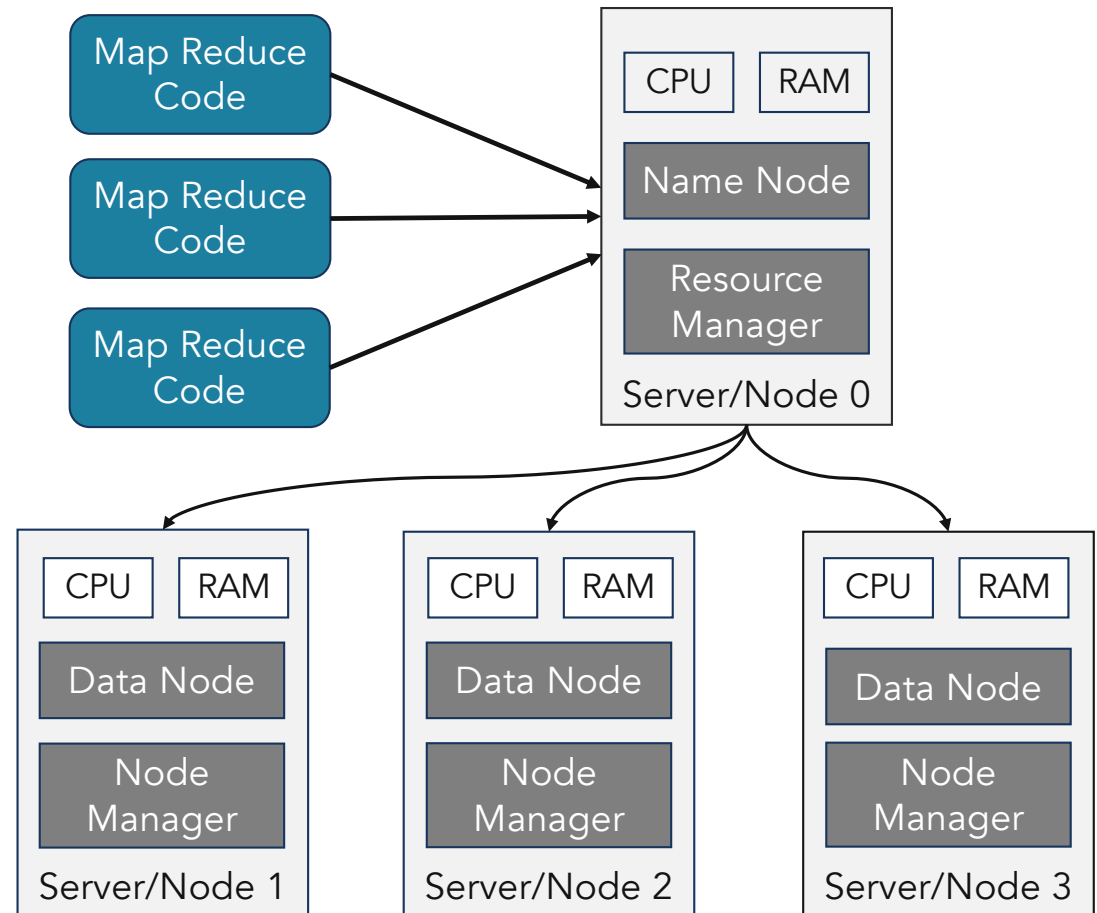
- Good for batch processing.
- Jobs run for hours or even days.
- Spark has overtaken MapReduce as the go-to solution for data processing.

Hands On – HDFS

- Launch an EMR cluster. Upload files to S3. Copy from local machine and S3 to HDFS and vice-versa.
- `hdfs dfs -copyFromLocal <local-file-path> <dest-on-hdfs>`
- `hdfs dfs -cat <path>`
- `hdfs dfs -copyToLocal <src-on-hdfs> <local-file-dest>`
- `hdfs dfs -moveFromLocal <local-file-sys> <dest-on-hdfs>`
- `hdfs dfs -cp <src-on-hdfs> <dest-on-hdfs>`
- `hdfs dfs -mv <src-on-hdfs> <src-on-hdfs>`
- `hadoop distcp hdfs://<src> <dest-file-system>`
- `s3-dist-cp --src <hdfs or s3> --dest <hdfs or s3>`
- `s3-dist-cp --src s3://<uri> --dest hdfs:///<dir>`
- `s3-dist-cp --src hdfs:///<dir> --dest s3://<uri>`

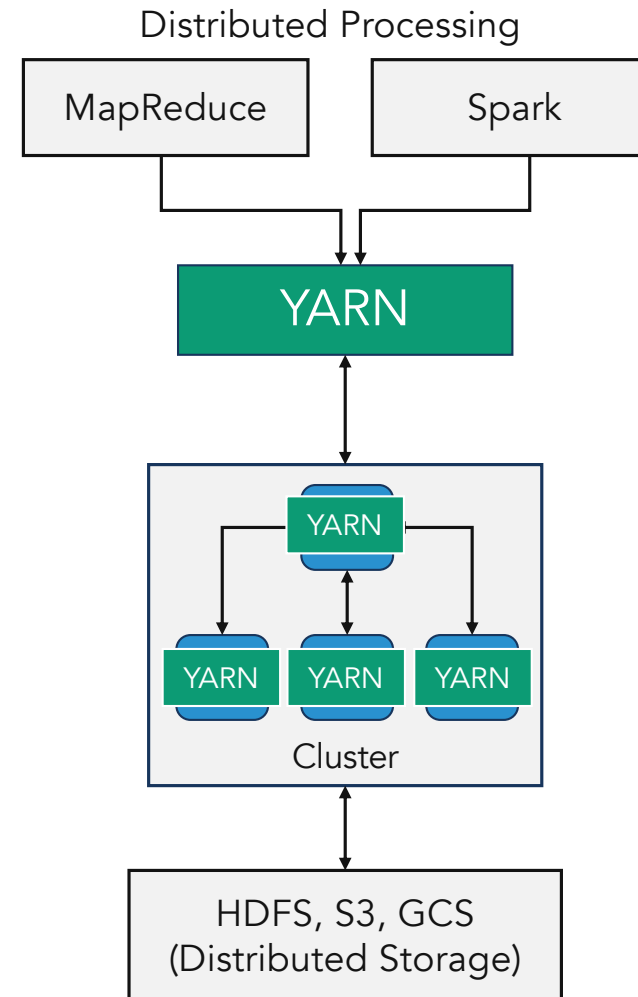
YARN

- Yet Another **Resource Negotiator**.
- Supports multiple distributed processing frameworks (including Spark).



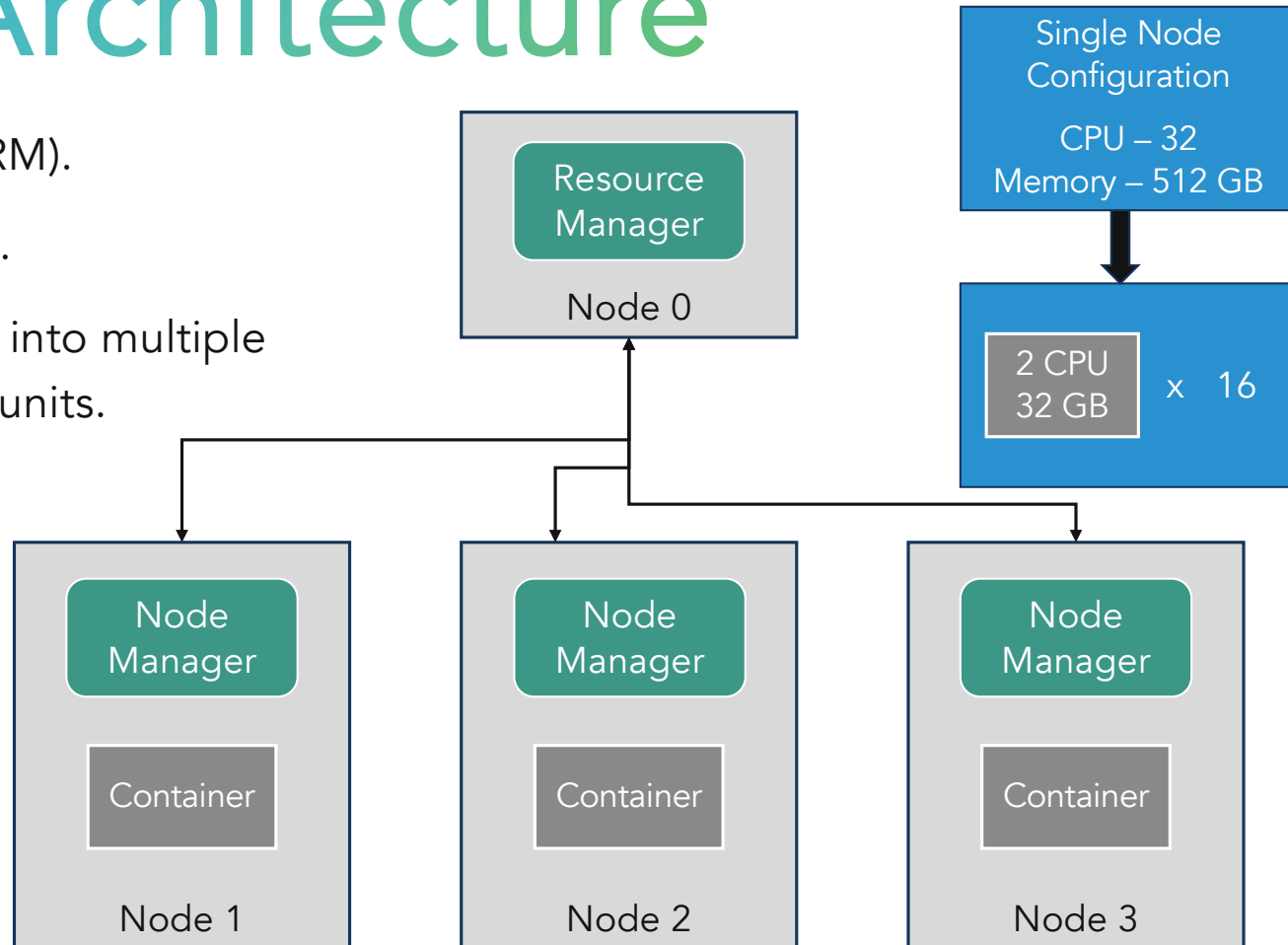
YARN

- Yet Another **Resource Negotiator**.
- Supports multiple distributed processing frameworks (including Spark).
- YARN finds resources (compute) to run the application.

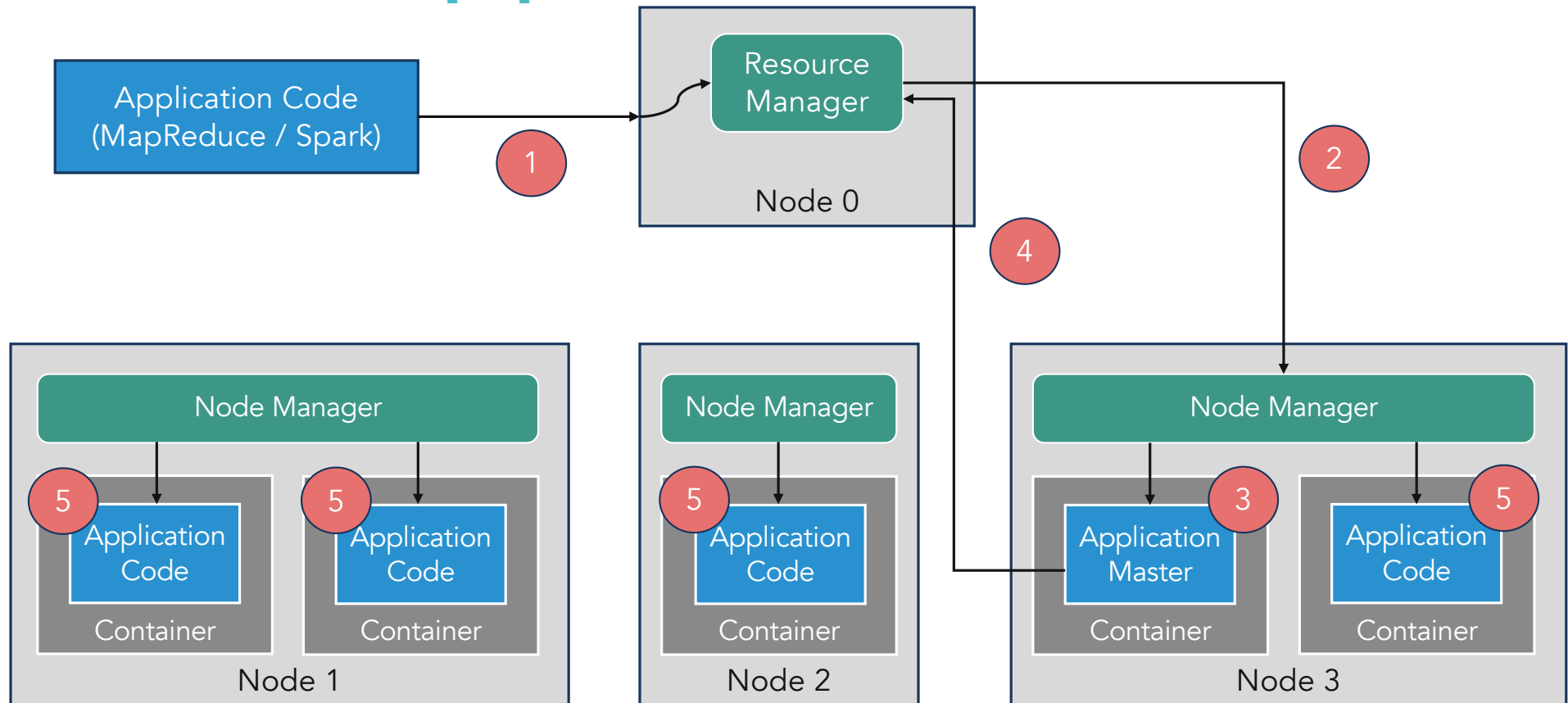


YARN Architecture

- Resource Manager (RM).
- Node Manager (NM).
- Breaks a single node into multiple small computational units.
- Containers.
- RM – Manages the use of resources across cluster.
- NM – Launch & Monitor containers.



YARN Application Flow



YARN Configuration

- YARN configuration file : yarn-site.xml

Configuration Variable	Description
yarn.scheduler.maximum-allocation-mb	Max memory that can be allocated to a container
yarn.scheduler.maximum-allocation-vcores	Max vCPU that can be allocated to a container
yarn.scheduler.minimum-allocation-mb	Min memory for a container
yarn.scheduler.minimum-allocation-vcores	Min vCPU for a container
yarn.nodemanager.resource.memory-mb	Allocated memory for a container
yarn.nodemanager.resource.cpu-vcores	Allocated vCPU for a container
yarn.scheduler.increment-allocation-mb	Increment step for container memory
yarn.scheduler.increment-allocation-vcores	Increment step for container vCPU

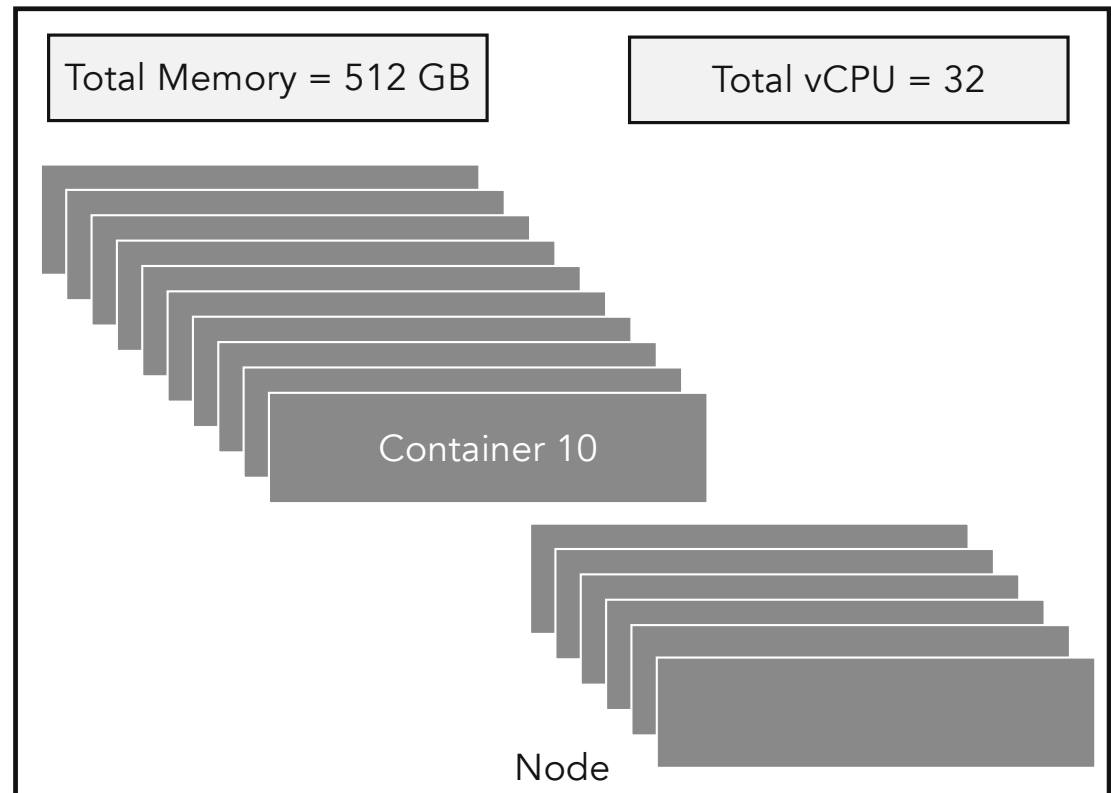
YARN Configuration

yarn.scheduler.minimum-allocation-mb = 16 gb
yarn.scheduler.maximum-allocation-mb = 64 gb

yarn.scheduler.minimum-allocation-vcores = 1
yarn.scheduler.maximum-allocation-vcores = 4

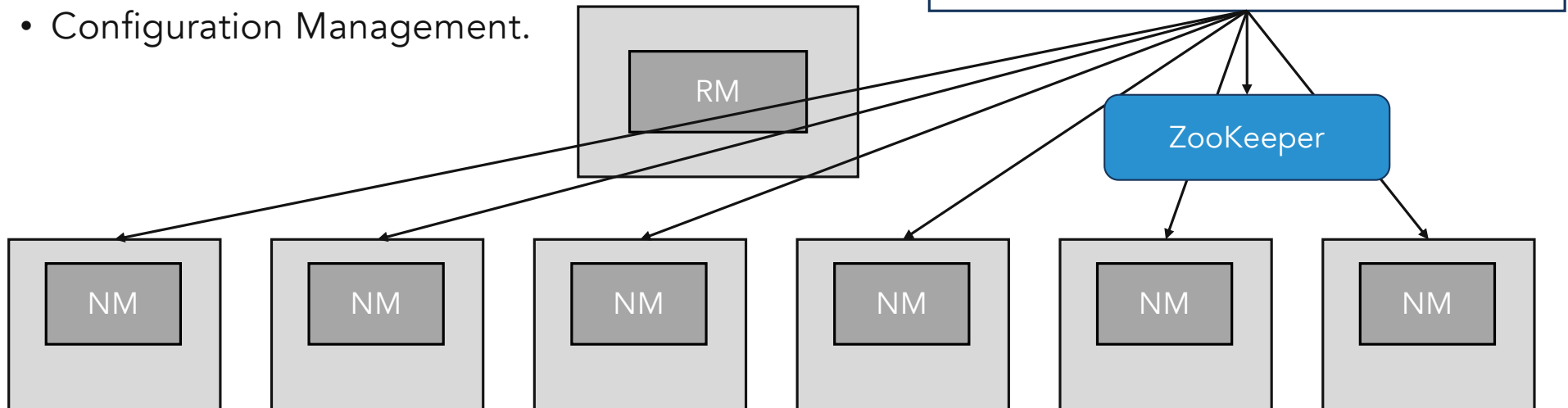
yarn.nodemanager.resource.memory-mb = 32 gb
yarn.nodemanager.resource.cpu-vcores = 2

Available vCPU = 12
Available RAM = 192 gb



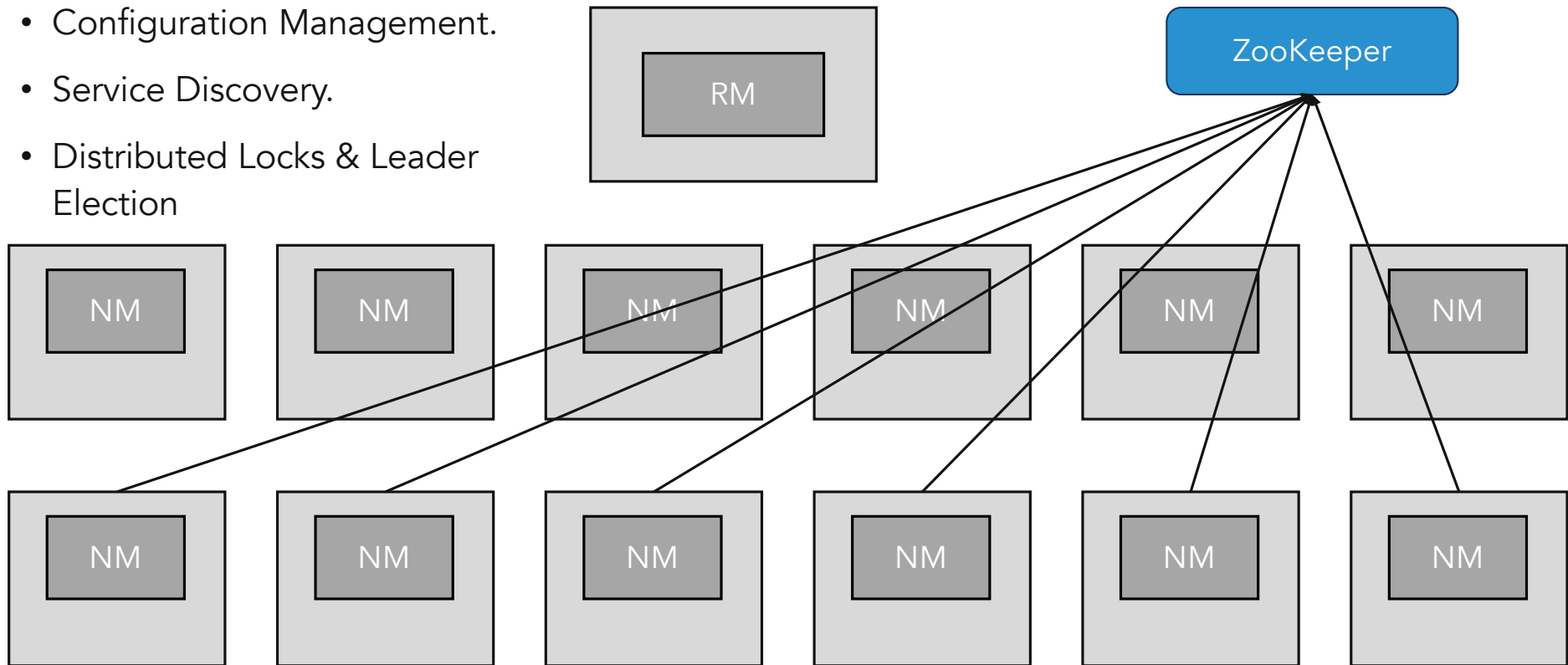
ZooKeeper

- Configuration Management.



ZooKeeper

- Configuration Management.
- Service Discovery.
- Distributed Locks & Leader Election

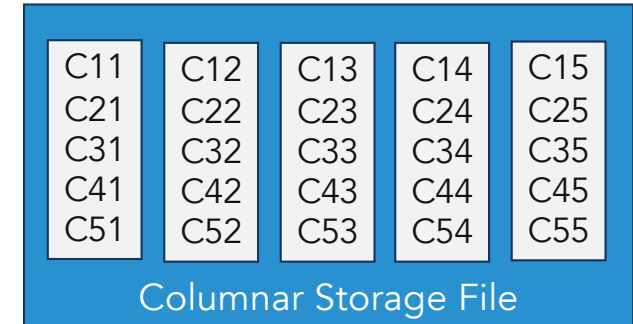
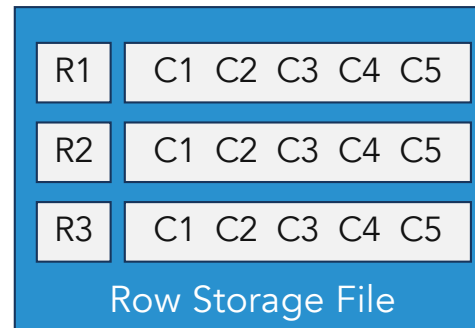




Big Data File Formats

Parquet

- Open-source File Format.
- Columnar Storage.
- Optimized for OLAP and Data Processing.
- Automatic Compression.
 - High Compression Ratio. 1 TB can be compressed to 100-200 GB.
 - Reduced IO.
- Metadata is stored along with the file.
- WORM use cases.

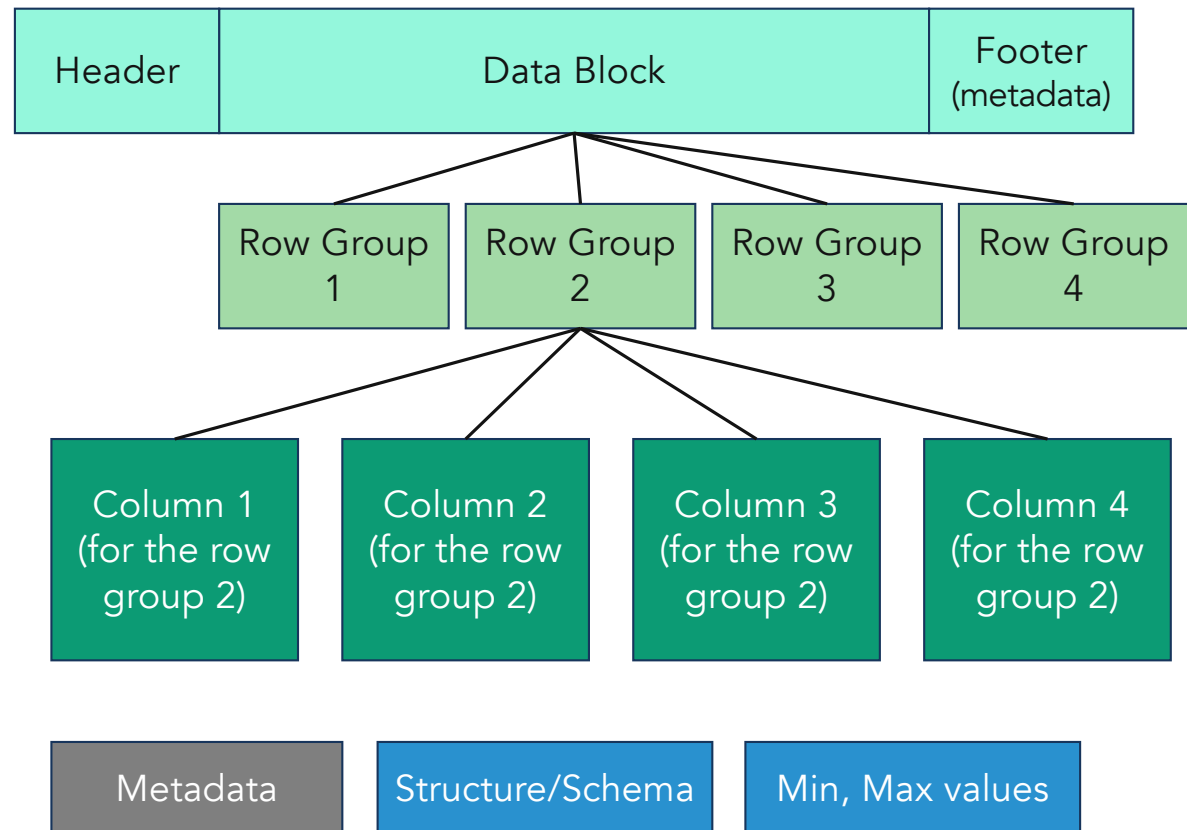


1 TB = 8192 Blocks = 8192 IOs

200 GB = 1600 Blocks = 1600 IOs

Parquet File Architecture

- Hierarchical.
- Header, Data Block, Footer.
- Row Group – Collection of a set of rows. Actual data block (128 MB).
- Metadata helps in reducing the no. of block reads.
- File metadata is used by readers to find the interested column chunks.
- Column chunks are read sequentially.



Avro

- Open-source File Format.
- Row Oriented Storage.
- Suitable for OLTP workloads.
- Automatic Data Compression.
- Schema is stored in JSON.
- Header.
- Data Blocks (16 KB).



Summary

- Big Data & its technologies
- Hadoop – MapReduce & HDFS
- YARN
- ZooKeeper
- File Formats – Parquet & Avro

The background of the slide is a monochromatic blue-toned photograph of a mountain range. The mountains are layered, with some peaks shrouded in mist or low clouds, creating a sense of depth. The sky is a lighter shade of blue, and the overall atmosphere is serene and calm. The text 'Thank You !!!' is centered in the middle of the image in a bright cyan color.

Thank You !!!